

EDS Lab Assignments

Practical1

Name: Prithviraj Thakur

PRN : 202501040026

Calculate Momentum

Problem Statement:

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: $p = m \times v$ where:

m is the mass of the object (in kilograms).
 v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
mass = float(input())
velocity = float(input())
momentum = mass * velocity
print(f"{momentum:.2f} kgm/s")
```

Output:

The screenshot displays a code editor interface with a file named '1.1.1. Calculate Momentum'. The code is as follows:

```
1 m=float(input())
2 v=float(input())
3 p=m*v
4 print(f"{p:.2f}kgm/s")
```

Below the code, the test results are shown. The summary indicates that 2 out of 2 shown test cases passed and 2 out of 2 hidden test cases passed. The performance metrics are:

Metric	Value
Average time	0.002 s
Maximum time	0.003 s

The test cases are detailed below:

Test Case	Expected output	Actual output
Test case 1	5.0	5.0
	10.0	10.0
	50.00kgm/s	50.00kgm/s
Test case 2		

2. Conditional Calculation based on the Number of Digits

Problem Statement:

Write a Python program that accepts an integer as input. Depending on the number of digits in.

Constraints: $1 \leq n \leq 999$

Input Format:

- The input consists of a single integer.

Output Format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid"

Code:

```
import math
n = int(input())
sq = n * n
sq_root = math.sqrt(n)
cb_root = math.cbrt(n)
if n > 1 and n <= 9:
    print(f"{sq}")
elif n >= 10 and n <= 99:
    print(f"{sq_root:.2f}")
elif n >= 100 and n <= 999:
    print(f"{cb_root:.2f}")
else:
    print("Invalid")
```

Output:

The screenshot displays a code editor with the Python program on the left and its test results on the right. The program is designed to calculate the square, square root, or cube root of an integer based on the number of digits. The test results show that the program passed all 7 test cases (4 shown, 3 hidden).

Test Results Summary:

Test Case	Expected Output	Actual Output	Status
Test case 1	9	9	Passed
Test case 2	81	81	Passed
Test case 3			Passed
Test case 4			Passed
Test case 5			Passed
Test case 6			Passed
Test case 7			Passed

3. Days Between Two Dates

Problem Statement:

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD.
- The second line contains the second date in the format YYYY-MM-DD.

Output Format:

- An integer representing the number of days between the given dates.

Code:

```
from datetime import date
from datetime import datetime
date1 = input().strip()
date2 = input().strip()
d1 = datetime.strptime(date1, "%Y-%m-%d")
d2 = datetime.strptime(date2, "%Y-%m-%d")
difference = d2-d1
print(difference.days)
```

Output:

The screenshot displays a code editor interface for a Python program. The code calculates the number of days between two dates using the `datetime` module. The program takes two dates as input and prints the difference in days.

Code Snippet:

```
1 # from datetime import date
2
3 #write your code here...
4 from datetime import date
5 import datetime
6
7 date1_str = input()
8 date2_str = input()
9 y1,m1,d1 = map(int,date1_str.split("-"))
10 y2,m2,d2 = map(int,date2_str.split("-"))
11 date1 = datetime.date(y1,m1,d1)
12 date2 = datetime.date(y2,m2,d2)
13
14 difference = date2-date1
15 print(difference.days)
```

Test Results:

- Average time: 0.001 s, Maximum time: 0.002 s
- 2 out of 2 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test Case 1:

Expected output	Actual output
2014-07-02	2014-07-02
2014-11-02	2014-11-02
123	123

Test Case 2:

Expected output	Actual output
123	123

4. Reverse a Number

Problem Statement:

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Code:

```
n=int(input())
reverse=int(str(n)[::-1])
print(reverse)
```

Output:

The screenshot displays a code editor interface for a problem titled "1.1.4. Reverse a Number". The problem description asks to write a Python program to reverse the digits of a number and print the reversed number. The input format specifies a single integer, and the output format specifies a single integer representing the reversed number. A section for sample test cases is visible but empty.

The code editor shows the following Python code:

```
1 n=int(input())
2 reverse=int(str(n)[::-1])
3 print(reverse)
```

Below the code, the execution results are shown. The average time is 0.001 s and the maximum time is 0.001 s. The test results indicate that 2 out of 2 shown test cases passed and 3 out of 3 hidden test cases passed. The details for Test case 1 are as follows:

Expected output	Actual output
5367	5367
7635	7635

Test case 2 also passed with a time of 0 ms.

5. Multiplication Table

Problem Statement:

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

<number> x <multiplier> = <result>

Code:

```
n = int(input())
```

```
for i in range(1,11):
```

```
    print(n, "x", i, "=", n*i)
```

Output:

The screenshot displays a coding interface with the following components:

- Problem Statement:** "Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10."
- Input Format:** "The first line of input contains an integer that represents the number for which the multiplication table is to be printed."
- Output Format:** "Print the multiplication table for the given number in the format: <number> x <multiplier> = <result>"
- Note:** "The character 'x' is used in the output to represent multiplication. Refer to the visible test-cases for more clarity."
- Sample Test Cases:** A button labeled "Sample Test Cases" with a plus icon.
- Code Editor:** Contains the following Python code:

```
1 n = int(input())
2 for i in range(1,11):
3     print(n, "x", i, "=", n*i)
```
- Test Results:** Shows "2 out of 2 shown test case(s) passed" and "2 out of 2 hidden test case(s) passed". The performance metrics are: Average time 0.002 s, Maximum time 0.002 s, and 1.50 ms.
- Test Case 1:** Shows the expected output and actual output for the input 8. The output is:

```
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
```

1.2.1 Pass or Fail

Problem Statement:.

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer, the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Code:

```
def calculate(marks, total_courses):
    if any(mark<40 for mark in marks):
        return "Fail"
    total_marks = sum(marks)
    aggregate_percentage = ((total_marks)/(total_courses*100))*100
    if aggregate_percentage > 75:
        grade = "Distinction"
    elif aggregate_percentage >= 60:
        grade = "First Division"
    elif aggregate_percentage >= 50:
        grade = "Second Division"
    elif aggregate_percentage >= 40:
        grade = "Third Division"
    return (aggregate_percentage, grade)
num_courses = int(input())
marks = list(map(int,input().split()))
result = calculate(marks, num_courses)
if result == 'Fail':
    print("Fail")
else:
    aggregate_percentage, grade = result
    print("Aggregate Percentage:",f"{aggregate_percentage:.2f}")
    print(f"Grade: {grade}")
```

Output:

1.2.1: Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

```
1 n=int(input())
2 mark=list(map(int,input().split()))
3 fail=False
4 for i in mark:
5     if i<40:
6         fail=True
7         break
8 if fail==True:
9     print("Fail")
10 else:
11     agg=sum(mark)/n
12     if agg>75:
13         print("Aggregate Percentage:",f"{agg:.2f}")
14         print("Grade: Distinction")
15     elif agg>=60 :
```

Average time: 0.001 s, 1.40 ms | Maximum time: 0.002 s, 2.00 ms

5 out of 5 shown test case(s) passed
5 out of 5 hidden test case(s) passed

Test case 1	Expected output	Actual output
	4	4
	55 65 78 89	55 65 78 89
	Aggregate Percentage: 71.75	Aggregate Percentage: 71.75
	Grade: First Division	Grade: First Division

1.2.2 Fibonacci Series

ProblemStatement:

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

Code:

```
def fibonacci(n, a=0, b=1):  
    if n==0:  
        return a  
    else:  
        return fibonacci(n-1,b,a+b)
```

```
n=int(input())  
for i in range(n):  
    print(fibonacci(i) ,end=" ")
```

Output:

The screenshot displays a code editor interface for a problem titled "1.2.2. Fibonacci Series". The problem description asks for a Python program using recursion to print the first n terms of the Fibonacci series. The input format is a single integer n , and the output format is a single line of the first n terms separated by spaces. A "Sample Test Cases" button is visible.

The code editor shows the following Python code:

```
1 # def fibonacci(n):  
2 #     if(n<2):  
3 #         return n  
4 #     else:  
5 #         return fibonacci( n - 1 ) + fibonacci(n-1)  
6  
7  
8 # n = int(input())  
9 # for i in range(1, n + 1):  
10 #     print(fibonacci(i), end=" ")  
11  
12 def fibonacci(n):  
13     if(n<2):  
14         return n  
15     else:
```

The execution results show that the program passed all test cases. The performance metrics are:

Metric	Value
Average time	0.005 s
Maximum time	0.016 s

The test results indicate that 2 out of 2 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. The details for Test case 1 are as follows:

Test case	Expected output	Actual output
Test case 1	0 1 1 2 3	0 1 1 2 3

1.2.3 Pattern-1

Problem Statement:

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Code:

```
rows = int(input())
for i in range(1, rows+1):
    for j in range(i):
        print("* ",end = "")
    print()
```

Output:

1.2.3. Pattern-1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

```
1 n=int(input())
2 for i in range(1,n+1):
3     for j in range(1,i+1):
4         print("*",end=" ")
5     print("")
6
```

Average time: 0.001 s, 1.17 ms | Maximum time: 0.002 s, 2.00 ms

2 out of 2 shown test case(s) passed
4 out of 4 hidden test case(s) passed

Test case 1 (1 ms) Debug

Expected output	Actual output
*	*
* *	* *
* * *	* * *
* * * *	* * * *
* * * * *	* * * * *

1.2.4 Pattern-2

Problem Statement:

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Code:

```
n=int(input())
for i in range(1, n+1):
    for j in range(1, i+1):
        print(j,end=" ")
    print()
```

Output:

